



A TWO-DAY STORY

One meeting transcript in. A quarter of engineering out.

On Thursday we had a call with a prospect. We handed the transcript to Claude Code running the APIANT plugin. Two days later we had a client proposal, a multi-location integration built and tested branch by branch, a marketing engine configured, and a documentation site. This is what happened, and what it means.



PREPARED BY
APIANT · fred@apiant.com

SUBJECT
Claude Code + APIANT API
infrastructure

DATE
July 2026

THE SETUP

The whole input was a conversation

A 100+ location wellness brand wanted Mindbody data in Klaviyo. Their agency joined the call. We talked for an hour about locations, flows, and what their native integration could not do.

There was no spec document. No Jira epic. No architecture review. We took the transcript of that call, dropped it into Claude Code with the APIANT plugin connected, and started asking for things.

What follows is the actual output of the next two working days. Every item on the list exists: it is deployed, published, committed, or sitting on a desktop. None of it is a mockup.

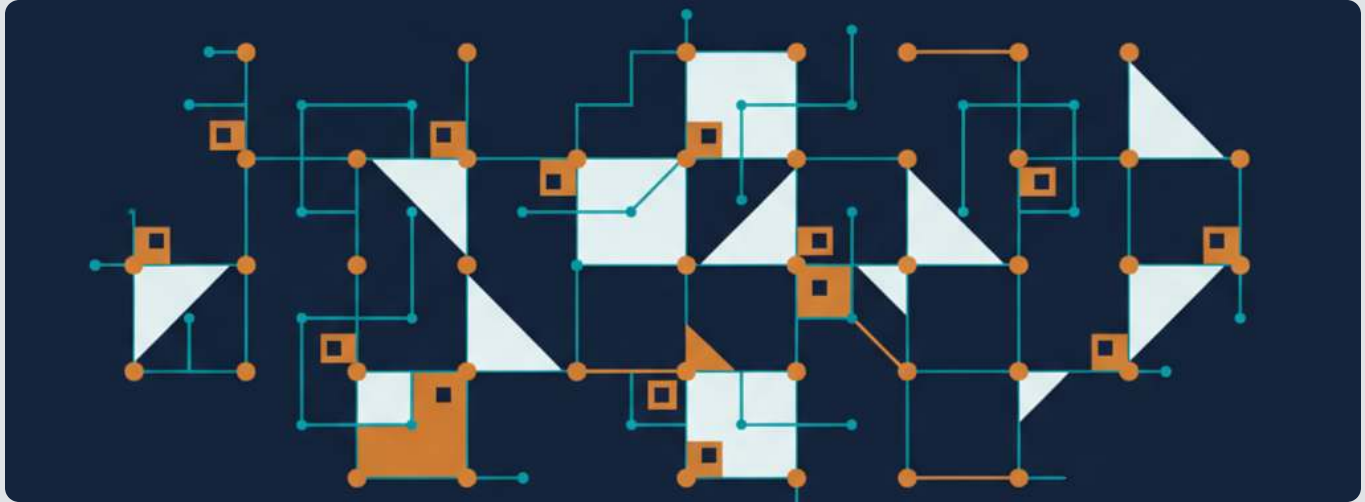


The nine things that came out of it

The integration itself, built apiant.com New automations, new subroutines, nine new Klaviyo API endpoints, 15 new event metrics. Engineered end to end, then tested, every branch.	A client proposal apiant.com 13-page PDF: verified pricing, live product screenshots, a customer testimonial, and the data story. Ready to send.
A full product audit Every automation in the fleet, every branch, verified at the logic level. Four real defects found and fixed.	A competitive analysis Property-by-property comparison against the native integration, corrected against live evidence, not marketing claims.
A migration plan A costed, sequenced plan to move 100+ locations off the native integration without breaking a single live flow.	A marketing engine 24 location-aware segments, welcome flows per studio, campaign drafts with designed emails. Configured, populated, verified.
A documentation site info.ariant.com Eight customer-facing pages published to info.ariant.com, from onboarding to a complete data reference.	A data dictionary google sheets Every synced property and event in one formatted spreadsheet, with working dropdowns, for customers and sales.
A product page apiant.com The complete public product page revised: functionality walkthrough, feature comparison against the native integration, positioning. Live on apiant.com.	

Why it works: the plugin, not the prompt

Anyone can ask an AI to write code. The difference here is that Claude Code was operating our platform, and building on it. The integration that shipped was engineered during those two days, not pulled off a shelf.



1 The plugin gives it hands

The APIANT plugin exposes our platform as tools: build an automation, inspect a running execution, deploy to a customer account, read the exact payload a webhook delivered. Claude does not guess what happened. It looks.

2 The infrastructure gives it leverage

It never hand-rolls an API client, a token refresh, or a rate limiter. Those are solved, once, inside APIANT. Everything above that layer it builds: the automations, the endpoints, the mappings, the branch logic. The plumbing is inherited. The product is built.

3 Everything is verifiable, so nothing is assumed

Every claim in this document was checked against the live system: executions read, events queried, segment counts confirmed. When a conclusion was wrong, the evidence said so, and it got corrected the same hour.

4 The work compounds

The audit fed the competitive analysis. The analysis fed the proposal. The proposal exposed a product gap, which became an issue, which became a fix. One conversation, one continuous thread.

The honest part. It was not autonomous magic. A human steered: caught an overstated claim, corrected a price, rejected a design, decided what shipped. The role changed from doing the work to directing it, and reviewing it.

The integration had to be built. So it was.

This is the part that is easiest to miss, and the most important. What the customer needed did not exist on Thursday: the automations, the Klaviyo API endpoints, the event schema all had to be created. Claude Code engineered them, then tested what it built, branch by branch, on live systems.

24

AUTOMATIONS IN THE
DEPLOYED FLEET

9

NEW KLAVIYO API
ENDPOINTS BUILT

15

NEW EVENT METRICS
DESIGNED

21

KLAVIYO WRITE STEPS
HARDENED

What was built

New automations

Instant handlers for appointment and class booking events, a cancellation chain, a **nightly self-healing sales sweep** that doubles as historical backfill, a contract-state reconciler, a CSV client importer, a fleet alert monitor, an API rate limiter, a Klaviyo package provisioner.

New core logic

Three subroutines carry the hard thinking: a **cross-site identity resolver** that never trusts a cached profile, a per-sale revenue processor, and a next-visit recompute engine that replaced two polling jobs with event-driven updates.

Nine new Klaviyo API endpoints

The connector did not have what the integration needed, so it was extended: **Create event, Create segment, Create flow, Subscribe profile**, plus their find and delete twins. Written against Klaviyo's current API revision, now reusable by every product in the catalog.

A new event schema

15 Klaviyo metrics designed from scratch: appointment, class, membership, and contract lifecycles, plus canonical **Placed Order and Ordered Product** revenue events with full line-item carts. Every event carries an idempotent unique id, so replays can never double-count.

Then the system tested the system

Every branch, executed

Not unit tests. Every automation, **every branch**, was driven with real webhooks and real API responses, and the execution logs were read step by step to verify what actually happened.

Adversarial drills

A poisoned-cache drill proved identity resolution **self-repairs** after an account switch. A full provisioner re-run proved zero duplicates. Sales replays proved revenue events cannot double-count.

Hardened for production

Retry-on-error on all **21 Klaviyo write steps**, with the rate-limit response verified on the server's retryable list. Errors park per client and clear on success instead of failing the run.

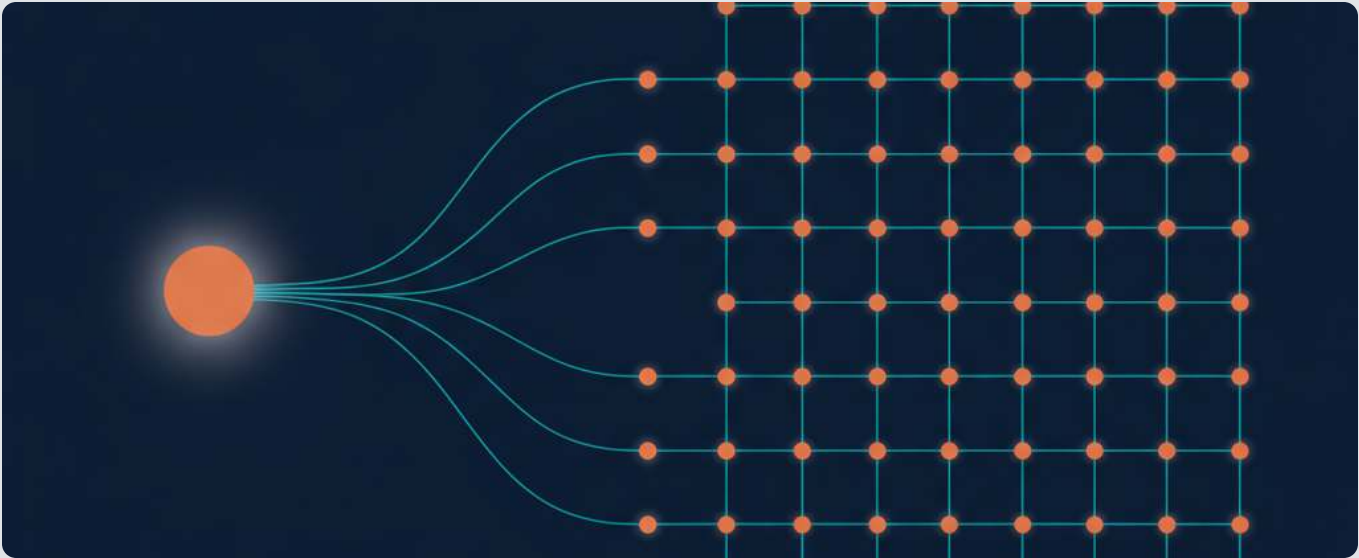
Defects found, fixed

The tests caught real bugs in the build (a dedupe key read but never saved, a per-line amount where a sale total belonged) and they were **fixed and re-verified the same session**.

THE HARD PART

A multi-location integration is where most tools break

Mindbody identifies a studio with a pair of numbers. Klaviyo needs a name a member will recognize. Multiply by 100+ studios, across several Mindbody sites, feeding one shared Klaviyo account, and this is the problem that kills integration projects.



One connection, then every location inherits the entire pipeline and its own marketing package.

In two days, on live systems, the integration was built to handle it. A managed naming table turns every site-and-location pair into the studio's real name, and every profile field, every event, and every audience is stamped with that name. Bringing a customer's next city online is a fixed sequence, not a project:

STEP	RESULT
Deploy	24 automations pushed to a new child account as a linked deployment, in one call
Configure	Settings run reads the new site's locations and names them automatically
Activate	The live pipeline switched on, webhooks registered, data flowing
Provision	The new location's segments and flows generated from the same naming table

What that means commercially. Adding a customer's 101st location is no longer a project. It is a connection and a settings run. That is the difference between an integration you sell once and a platform you scale on.

What this would have cost in people and weeks

Below is the same body of work, estimated the ordinary way: a full-time systems integrator, a technical writer, and a designer, at the rate real teams actually deliver.

DELIVERABLE	EST. HOURS
The integration itself: build, test, harden Two-way Mindbody and Klaviyo pipeline, multi-location, real-time events, historical backfill, rate limits, error handling, end-to-end QA. Two to four months of one full-time systems integrator.	320 - 640
Full fleet audit Every automation, every branch, verified at the logic level; 4 defects found and fixed	24
Klaviyo marketing engine 24 segments, flows per location, campaigns, designed emails, data cleanup across 157 profiles	30
Customer documentation site Eight published pages including a complete data reference	22
Client proposal PDF Pricing verified against live rate cards, product screenshots, layout, revisions	14
Migration plan for 100+ locations Compatibility design, cutover runbook, risk and rollback	14
Competitive parity analysis Property and event comparison, corrected against live evidence	10
Engineering hygiene 10 issues filed with reproductions, known-issues log, 46 commits	10
Data dictionary spreadsheet 70 rows, formatting, working dropdowns	7
Brand and creative assets Logo set, email artwork, campaign imagery	8
Conventional delivery	439 - 759 hours

~600 h

THE NORMAL WAY
(MIDPOINT)

3 - 5 mo

ONE FULL-TIME
INTEGRATOR

2 days

WHAT IT TOOK

~40×

COMPRESSION

The integration line is the industry benchmark, not a pessimistic one: a two-way, multi-location, real-time pipeline with backfill and rate-limit handling is a two-to-four-month engagement. Every other line assumes people who already know these systems intimately, and none of them price in coordination overhead or review cycles. The true number is higher.

What one conversation is now worth

The proposal that came out of Thursday's call is a real, priced, signed-ready document. Here is what it carries, using the numbers exactly as they were sent.

THE PROPOSAL AS SENT	VALUE
Fleet size Locations across the brand, one shared Klaviyo account	106 sites
Onboarding, one time \$50 per site, all 106 included, against \$54,000 list	\$5,300
Recurring, monthly \$50 per location per month, both pipelines, no contact-volume pricing	\$5,300/mo
Recurring, annualized Annual commitment, rate locked as the fleet grows past 150 and 200	\$63,600/yr
Year one contract value	\$68,900

Now put the two numbers side by side. Delivering an integration of this complexity the conventional way costs roughly 600 hours. At a blended \$125 an hour that is about **\$75,000 of labor** to win a deal worth **\$68,900 in year one**. The economics do not work. That is precisely why nobody builds these, and why the native integration stays free and shallow.

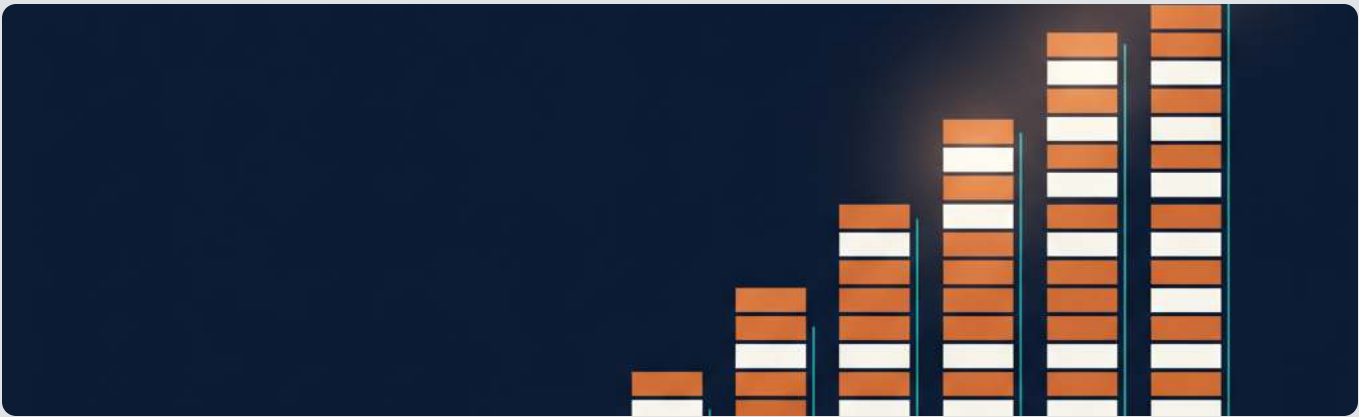
\$78k LABOR COST, THE OLD WAY	\$68.9k YEAR ONE REVENUE	~\$2k LABOR COST, TWO DAYS	97% YEAR ONE GROSS MARGIN
---	------------------------------------	--------------------------------------	-------------------------------------

That is the whole shift, in one line. The same deal that was **unprofitable to deliver** is now delivered at a **97% year-one margin**, and every year after it is close to pure recurring revenue. The integration stopped being the cost centre and became the product.

Blended rate of \$125 an hour is an assumption for illustration; substitute your own. The conclusion holds anywhere between \$80 and \$200 an hour.

And there is nothing special about this customer

Every multi-location brand on Mindbody has the same problem, the same native integration, and the same ceiling. The motion that produced this proposal is now repeatable in days, not quarters.



Every location added is \$50 once and \$50 a month, with zero incremental engineering.

Recurring revenue by fleet size, at the same partner rate

BRAND PROFILE	ONBOARDING	ANNUAL RECURRING
Regional operator, 25 locations	\$1,250	\$15,000
Growing franchise, 50 locations	\$2,500	\$30,000
The proposal on the table, 106 locations	\$5,300	\$63,600
National brand, 235 locations	\$11,750	\$141,000

The growth annuity

These brands add studios continuously. A brand opening two sites a month adds **\$14,400 of recurring revenue a year**, automatically, with no engineering work at all. Growth is billed, not built.

The catalog multiplier

Mindbody and Klaviyo is **one of eighteen** CRMConnect products. The same two-day motion, the same infrastructure, the same plugin, applies to every other pair in the catalog.

The integration is now a productized mini SaaS: ten customers at this profile is roughly \$640K ARR, and every integration built this way deploys to a new customer in hours using the plugin.

Integration complexity is no longer the constraint

For twenty years, the cost of connecting two systems scaled with how complicated they were. Multi-location. Real-time events. Historical backfill. Rate limits. Field mapping across a hundred sites. Each one added weeks.

That relationship just broke.

Not because a model wrote clever code in a vacuum. The plumbing (authentication, retries, throttling, webhook delivery, deployment to customer accounts, execution history you can actually read) was already solved inside our infrastructure as tested modules. **On top of that layer, Claude Code built the integration itself:** the automations, the Klaviyo API endpoints, the event schema. Then it tested every branch of what it built against the live system before anything shipped.

An hour of conversation became a quarter of engineering output. The work that used to define the project is now the part nobody thinks about.

Two days from a meeting transcript to an integration built and tested, a proposal on the desk, a marketing engine configured, and the documentation to support it. Not a prototype. The real thing, on real systems, verified.

And the commercial consequence is not subtle. An integration that cost more to build than it earned in its first year now carries a **97% year-one margin**, with a **\$63,600 annual recurring** proposal already in the customer's hands and seventeen more products in the catalog waiting for the same treatment.

This is what our API infrastructure makes possible. Any complexity, any number of locations, any depth of data, now trivial to build and, just as importantly, trivial to explain, document, and sell.

The next customer conversation starts the same way. So does the next integration.